

L03 Rigid-body state, and two views

A pose is not a list of numbers you can add, and a second camera is not a second chance to measure depth.

Rigid-body state, and two views

L03 · Unit 0

A pose is not a list of numbers you can add. A second camera is not a second chance to measure depth.

We stopped last week on the definition of a group. The first half of today uses it: what $[R \mid t]$ is, where the arithmetic on it happens, and where its uncertainty lives. The second half asks what two cameras tell you before anything has been matched.

Today

1. **Rotations as a group**, poses as a manifold, and where the arithmetic on them happens.
2. **Uncertainty** in the tangent space, the lever arm, and the edge of a chain that drifts.
3. **Epipolar geometry**: what a second view constrains before you have matched anything.
4. **Features**: what makes a point matchable at all.

Notes: Module **A1** and **A6** for the first half, **A2** and **A5** for the second. Lab 1 uses every piece of this.

What a group is

where we stopped

A **group** is a set of objects together with one way of combining two of them, obeying four rules.

- **Closure.** Combine two members and you get a member, never anything else.
- **Associativity.** $(ab)c = a(bc)$, so a chain needs no brackets.
- **Identity.** One member e that changes nothing: $ae = ea = a$.
- **Inverse.** Every member a has a partner a^{-1} with $aa^{-1} = e$.

Set and operation	Group?	Why
Integers, $+$	Yes	identity 0, inverse $-a$
Integers, $\times$	No	2 has no integer inverse
Shuffles of a deck	Yes	undo any shuffle, do nothing = identity
Rotations, composition	Yes	the one we need

SO(3): rotations

$$\text{SO}(3) = \left\{ R \in \mathbb{R}^{3 \times 3} : R^T R = I, \det R = +1 \right\}$$

■ Count the degrees of freedom: 9 entries, how many constraints?

$R^T R = I$ is a *symmetric* matrix equation, so 6 independent constraints rather than 9. Nine minus six leaves **three**. $\det R = +1$ picks a connected component and removes no dimensions.

Three numbers of information, stored in nine, held on a curved surface.

SE(3): poses

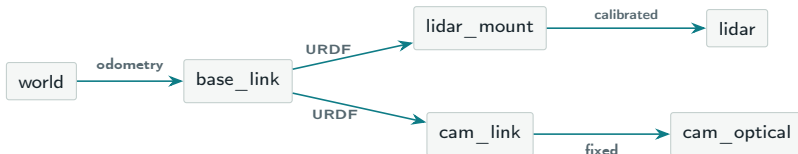
$$T = \begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}, \quad R \in \text{SO}(3), \quad t \in \mathbb{R}^3$$

- Six degrees of freedom: three from R , three from t .
- Composition is matrix multiplication, which is why we padded points to four components.
- Inverse in closed form: $T^{-1} = \begin{bmatrix} R^\top & -R^\top t \\ 0^\top & 1 \end{bmatrix}$.

Trap

Never call `np.linalg.inv` on a pose. It is slower, and it silently drifts off the manifold.

Frames and transform trees



- Each edge is a pose. Reading a chain means multiplying along the path: $T_{wc} = T_{wb}T_{bc}$.
- It is a **tree**: exactly one path between any two frames, so a lookup is unambiguous.

Notation: T_{wc} maps points **from** the camera **into** the world, $p_w = T_{wc} p_c$. Adjacent subscripts cancel.

The four edges fail in four different ways

Edge	Where the number comes from	What kind of error it has
world → base	Odometry or SLAM	Grows without limit until a loop closes.
base → mount	A CAD model, via the URDF	Fixed and wrong, if the part was fitted by hand.
mount → lidar	You estimated it	Fixed and uncertain. Bends the whole map.
cam → optical	A convention someone chose	Either exactly right or an axis flip.

Only the first one changes over time. The other three are constants, which makes them easy to forget and impossible to notice.

You cannot add two rotations

You want to say: *my estimate is $\bar{R}$, and the truth is a little off*. You cannot write $R = \bar{R} + \Delta R$, because the sum is not a rotation.

■ **Average a 0° and a 180° rotation about z**

$$\frac{1}{2} \left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The average of two perfectly good orientations collapses the x and y axes to nothing. Its determinant is 0, so it has stopped being an orientation rather than become an inaccurate one.

What a vector space gives you

A **vector space** is a set where you can add any two members, scale any member, and stay inside. $\mathbb{R}^3$ is one; $\text{SO}(3)$ is not. Calculus is built from those two operations:

$$\frac{\partial f}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x + he) - f(x)}{h}$$

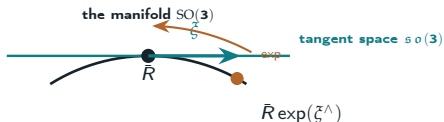
- f is whatever you are differentiating, e a direction to move in and h a small step along it. Forming $x + he$ needs addition *and* scaling, and the result has to remain a legal input to f .
- On $\text{SO}(3)$ that step leaves the set, so the limit is asking about a point that does not exist.
- Gradient descent, Gauss–Newton and Kalman updates all *add a correction*. None can run here.

No addition, no derivative. No derivative, no optimiser.

Flatten it locally: the tangent space

$SO(3)$ is a curved surface sitting in $\mathbb{R}^{3 \times 3}$. At any point on it there is a flat plane that touches it, the **tangent space**, and a plane *is* a vector space. Calculus works there.

■ The curve, the plane touching it at $\bar{R}$, and $\exp$ carrying ξ back down onto it



At the identity that plane is $\mathfrak{so}(3)$: the **skew-symmetric** matrices, $A^\top = -A$. Three free numbers, and nothing tying them together.

Three numbers, not nine

the hat map

■ Write out a skew-symmetric 3×3 and count what is free

$A^\top = -A$ zeroes the diagonal and makes the lower triangle minus the upper. Three entries are free:

$$\omega^\wedge = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix} \longleftrightarrow \omega = (\omega_1, \omega_2, \omega_3)$$

You have met it: $\omega^\wedge x = \omega \times x$, the cross-product matrix, back this afternoon inside E.

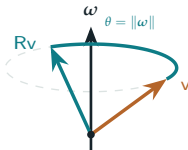
The tangent space at the identity is $\mathbb{R}^3$ in a matrix costume: three numbers you may add and scale.

Where exp comes from

■ Turn at a constant ω , and integrate for one second

Turning at a constant angular velocity means the rotation obeys a linear differential equation, and a linear ODE with constant coefficients is solved by an exponential:

$$\dot{R} = \omega^\wedge R, \quad R(0) = I \quad \implies \quad R(t) = \exp(t \omega^\wedge)$$



Read it as: spin about $\omega / \|\omega\|$ at $\|\omega\|$ rad/s for one second, then stop. So $(0, 0, \pi/2)$ is a 90° turn about z . Log runs it backwards.

$\mathfrak{so}(3)$ and $\mathfrak{se}(3)$

what the flat space contains

	SO(3)	SE(3)
Group element	$R, 3 \times 3, R^T R = I$	$T, 4 \times 4, \text{ a rotation and a translation}$
Degrees of freedom	3	6
Tangent space at I	$\mathfrak{so}(3)$	$\mathfrak{se}(3)$
whose elements are written as a vector	skew-symmetric 3×3 $\omega \in \mathbb{R}^3$	4×4 : a skew block and a column $\xi = (v, \omega) \in \mathbb{R}^6$

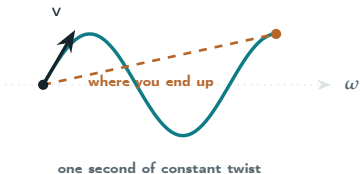
$$\xi^\wedge = \begin{bmatrix} \omega^\wedge & v \\ 0^\top & 0 \end{bmatrix} \in \mathfrak{se}(3)$$

Both are **vector spaces**: nothing ties the entries together, so you may add and scale them. That is the entire reason for going there.

A twist

what exp does to a six-vector

$\|\text{Log}(R_1^\top R_2)\|$ is the angle between two rotations, in radians. That is what a rotation error *is*, and it is the first thing the tangent space buys you.

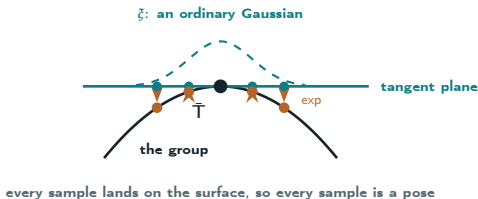


exp of a twist is where you end up after moving that way for one second, **turning while you translate**. The path is a helix, a screw.

So v is a velocity, not a displacement. The chord is shorter than the path, and points elsewhere.

Uncertainty lives in the tangent space

A pose holds 6 degrees of freedom in 12 numbers, so a spread over those 12 numbers is meaningless: most directions leave the surface entirely. Put the **mean on the surface** and the **noise on the flat plane touching it there**, the same plane the twist lives in.

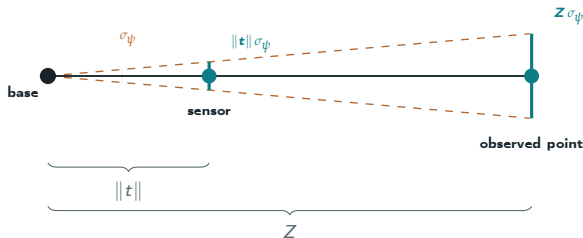


- On the plane it is an ordinary Gaussian over six numbers, so it has an ordinary 6×6 covariance. This is the number your SLAM system reports.
- Pushed down by exp it becomes a spread of poses, each one valid by construction.

Moving uncertainty between frames

the lever arm

An uncertainty is always quoted in some frame. Describe the same uncertainty from a different frame and it changes shape, because the conversion mixes **rotation into position**: a small angular error at the base becomes a position error out where the sensor is.



It grows with the mounting offset $\|t\|$, and then again with the range Z to whatever you observed. The same small angle, multiplied twice.

Half a degree, in centimetres

■ Base yaw error to sensor position error, then to point error

Base yaw uncertainty $\sigma_\psi = 0.5^\circ = 8.73 \times 10^{-3}$ rad, sensor mounted $\|t\| = 2$ m from the base:

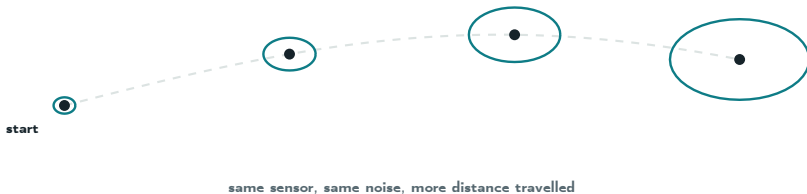
$$\sigma_p \approx \|t\| \sigma_\psi = 2 \times 8.73 \times 10^{-3} = 1.7 \text{ cm}$$

A point observed 20 m out inherits the same angular error again: $20 \times 8.73 \times 10^{-3} \approx 17$ cm, before the sensor's own noise contributes anything.

Half a degree at the base is seventeen centimetres at the far wall.

Chains, and the edge that drifts

A chain of estimated poses is the normal case: odometry, then a mount, then a calibration. Each link carries its own uncertainty into the final frame, and they accumulate.



- Every edge in the tree has a bounded error except world $\rightarrow$ base. That one integrates incremental motion, so it grows without limit.
- The error is in the integration, not the measurement. A better sensor does not fix it. Closing a loop does.

PnP, framed properly

Given 3D points **in a map you already have**, and their projections in the image you are looking at right now, find $[R \mid t]$. Both halves of today's lecture meet here.

$$\arg \min_{R \in \text{SO}(3), t \in \mathbb{R}^3} \sum_i \|u_i - \pi(K [R \mid t] \tilde{X}_i)\|^2$$

- The reprojection error from the first half, minimised over a pose instead of over P . As before, π is the perspective divide, $(a, b, c) \mapsto (a/c, b/c)$, turning the product back into a pixel.
- Six unknowns, two equations per correspondence, so **three points** is the minimum. Three points admit up to four valid solutions; a fourth disambiguates.

Wrapped in RANSAC, this is the answer to “where am I?”

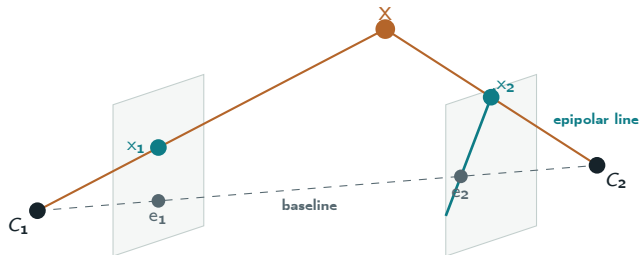
Why we optimise on the manifold

An optimiser repeatedly asks *which small motion would reduce the error*, then applies it. Both halves of that sentence are where today's second half arrives at once.

- **What it solves for** is a twist: six unconstrained numbers, rather than twelve matrix entries with six constraints between them. Unconstrained is what makes the step computable.
- **How it applies the answer** is by composing, not adding. The current estimate is moved along that twist, the way exp moves you along the surface.
- So the estimate is a valid pose at every iteration. Nothing to re-orthogonalise, nothing that slowly slides off the manifold.

Add a correction to a pose and you get something that is not a pose. Compose one and you always do.

Epipolar geometry



- The **baseline** joins the two camera centres. It and X span the **epipolar plane**, which each image sees edge-on, as the **epipolar line**.
- The **epipoles** e_1, e_2 are where the baseline pierces each image, and every epipolar line passes through them.

The epipolar constraint

■ Write coplanarity as a triple product, then read off the matrix

Work in **normalised** coordinates (K^{-1} already applied), with camera 2's pose relative to camera 1 given by (R, t) . The two rays and the baseline all lie in the epipolar plane, and three vectors are coplanar exactly when their triple product vanishes:

$$x_1 \cdot (t \times R x_2) = 0 \quad \implies \quad x_1^T \underbrace{t^{\wedge} R}_E x_2 = 0$$

The hat operator from Monday, doing real work. The cross product is what turns coplanarity into a matrix.

The essential matrix

$$E = t^{\wedge} R \quad x_1^T E x_2 = 0$$

- $E x_2$ **is** the epipolar line in image 1. That is the line's coefficient vector, literally.
- Rank 2, and its singular values are $(\sigma, \sigma, 0)$: a strong structural constraint.
- **Five** degrees of freedom: 3 rotation + 3 translation, minus 1 for overall scale.

Trap

E knows the direction of t , never its length. Scale is gone again, for exactly the reason from L01.

Why K^{-1} appears in F

■ Substitute $x = K^{-1}u$ into the epipolar constraint

The constraint is a statement about **rays**, and what a camera hands you is pixels. The two are related by the intrinsics: $u = Kx$, so $x = K^{-1}u$. Put that into $x_1^T E x_2 = 0$, one camera at a time:

$$(K_1^{-1}u_1)^T E (K_2^{-1}u_2) = 0 \quad \implies \quad u_1^T \underbrace{K_1^{-T} E K_2^{-1}}_F u_2 = 0$$

F is E with a change of coordinates wrapped around it, one factor per camera. Nothing extra is measured. The same zero is written in pixels instead of rays.

Correspondences alone give you F . Getting E out of it costs you the intrinsics.

The fundamental matrix

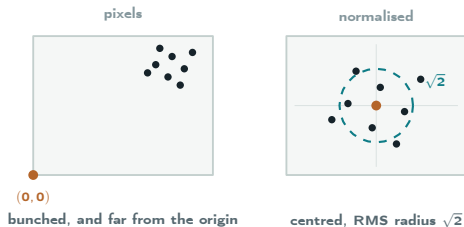
when you do not know K

$$u_1^\top F u_2 = 0, \quad F = K_1^{-\top} E K_2^{-1}$$

- Works directly on **pixels**. Seven degrees of freedom: 9 entries, minus scale, minus $\det F = 0$.
- E is the calibrated case, F the uncalibrated one. On a robot you almost always have K, so you almost always want E.
- F stays useful as a check, because you can estimate it without trusting your calibration.

Normalise first

or the SVD hands back noise



- Pixel coordinates run to $O(10^3)$, so entries of A span $u^2 \sim 10^6$ down to 1. The condition number is enormous.
- **Translate** so the centroid sits at the origin, then **scale** so the mean squared distance from it is 2, an RMS radius of $\sqrt{2}$.
- Matches bunched in one region of the image are the worst case.

One correspondence, one row

■ Turn $u_1^T F u_2 = 0$ into one row of a linear system

Expand with $u_1 = (u_1, v_1, 1)$, $u_2 = (u_2, v_2, 1)$ and collect the nine entries of F into a vector f :

$$\begin{bmatrix} u_1 u_2 & u_1 v_2 & u_1 & v_1 u_2 & v_1 v_2 & v_1 & u_2 & v_2 & 1 \end{bmatrix} f = 0$$

One equation per correspondence, unlike P , which gave two. Eight of them determine f up to scale, so stack the rows into $Af = 0$.

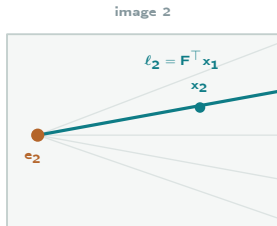
Five steps, in order

1. **Normalise** both point sets, and keep the two transforms T_1, T_2 .
2. **Build** A , one row per correspondence.
3. **Solve** $Af = 0$ by SVD. f is the smallest right singular vector, reshaped into $\hat{F}$.
4. **Enforce rank 2**: take the SVD of $\hat{F}$, set its smallest singular value to zero, recompose.
5. **Denormalise**: $F = T_1^T \hat{F} T_2$, undoing step 1.

Why step 4 is needed

The linear solution is not rank 2, so its epipolar lines do not meet at a common epipole. The next slide says where that requirement comes from.

Epipolar lines, and why F is rank 2



$Fe_2 = 0$, so F is rank 2

■ Every epipolar line contains the epipole

Fu_2 is the line in image 1 and $F^T u_1$ the line in image 2. The epipole lies on **every** one of them, whatever u_1 was:

$$e_2^T (F^T u_1) = 0 \quad \forall u_1 \implies Fe_2 = 0$$

A matrix with a null vector is singular, so $\det F = 0$ and F has rank 2. Likewise $F^T e_1 = 0$: the two epipoles are the null vectors of F and of its transpose.

When two views tell you nothing

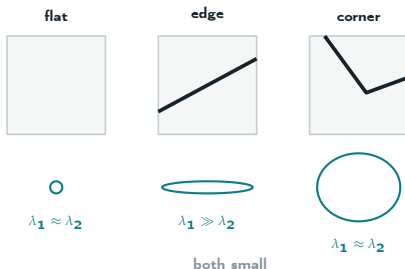
degeneracy

- **Pure rotation.** $t = 0$, so $E = 0$. No parallax, no depth, at any baseline you like.
- **A planar scene.** All correspondences satisfy a homography $u_1 \sim H u_2$, and F becomes underdetermined.
- Both are common: a robot turning in place, and a robot facing a wall or a floor.

Used by

A real front end fits **both** F and H , compares how well each explains the data, and picks. ORB-SLAM's initialiser is exactly this, and it is why it does not fall over in a corridor.

Which points can be matched?



- **Flat.** Every nearby patch looks the same, so there is no information in any direction.
- **Edge.** You can localise across the edge but slide freely along it. The aperture problem.
- **Corner.** The patch changes under motion in *every* direction, so it is localisable in two dimensions.

What you should be able to do

- Say why a pose cannot be corrected by addition, and where the correction goes instead.
- Derive the epipolar constraint from coplanarity, and say what E has that F does not.
- Show where each K^{-1} in F comes from.
- List the five steps of the eight-point algorithm, and say what breaks if you drop step 1 or step 4.
- Show that $Fe_2 = 0$, and say why that makes F rank 2.

Notes: Modules **A1**, **A6**, **A2**, **A5**. Szeliski §7.

Next time

L04 · Robust estimation, and 3D sensing

You can now write down the geometry of two views and say which points are worth matching. Monday finds the matches, then fits the geometry to them when a third are wrong.

- Features: invariance, the feature transform, image pyramids, and matching as a distance.
- RANSAC, its iteration count, and why the minimal sample size sits in an exponent.
- Triangulation, then structure from motion and bundle adjustment, step by step.
- Visual SLAM, then LiDAR and the difference between metric and relative depth.

Both diagnostic quizzes are due Sun Sep 27. Quiz 1 covers today and the first half of Monday.